

Partner Portal / Admission CRM

High-Level Design (HLD) & Low-Level Design (LLD)

Stack: MongoDB · Express.js · React/Next.js · Node.js (MERN) · Source: Developer Handover Pack v1.0, SRS v1.0 · v1.0

PART A — HIGH-LEVEL DESIGN (HLD)

System boundaries, layers, data flow, technology choices — the "what" and "why"

System Architecture

HLD-ARCH

TIER 1 — CLIENT (role-based, per tenant)

- Super Admin
- Partner Admin
- Team Member
- Support

HTTPS ▼

TIER 2 — Edge / Delivery · Vercel or Netlify

- Static Hosting
- CDN
- SSL

/api/v1/... ▼

TIER 3 — Application · Node.js + Express (modular monolith)

- Auth MW
- RBAC MW
- Tenant-Scope MW
- Feature Modules

Mongoose ODM ▼

TIER 4 — Data Layer

- MongoDB Atlas
- Redis
- S3 / Cloudinary

webhooks ▼

TIER 5 — External Integrations (Phase 3+)

- Payment Gateway
- WhatsApp Business API
- Email / SMS

ARCHITECTURAL STYLE

Modular monolith — one Express app, independent feature modules communicating via internal service functions

Multi-tenant by partnerId — Super Admin has unrestricted cross-tenant visibility

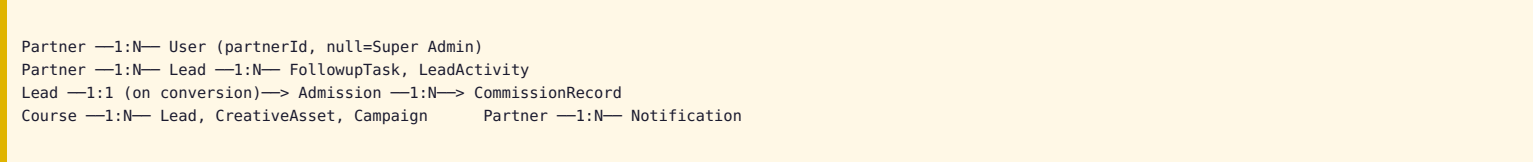
DESIGN GOALS

Server-side tenant scoping — never just hidden in the UI

Module boundaries extractable into microservices later without a rewrite

Every sensitive action audit-logged

Collection overview



Collection	Purpose
Partner	Tenant root — one document per academy
User	Login for all four roles; passwordHash never returned
Lead	Central pipeline entity — status: new → contacted → follow_up → demo → admitted/lost
Admission	Created on lead conversion; drives commission
CommissionRecord	earnedAmount / pendingAmount / paidAmount, payout lifecycle
AuditLog	Write-once; Super Admin read-only access

Lead-to-admission workflow

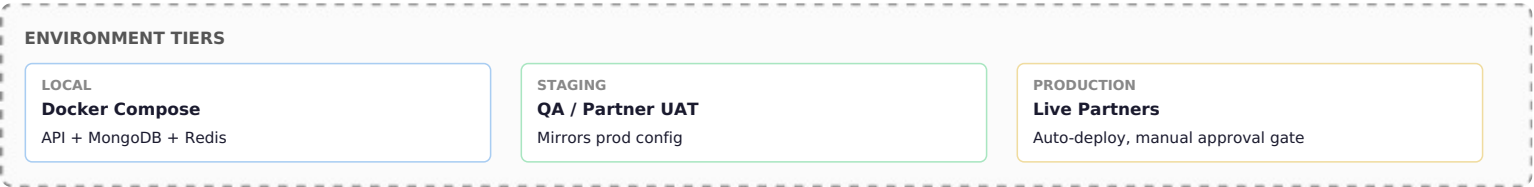


Role scope summary

Role	Scope	Representative permissions
Super Admin	Global	partner:approve, role:manage, report:view_all, audit:view
Partner Admin	Own partner	lead:*, admission:*, commission:view, team:manage
Team Member	Assigned records	lead:update_assigned, followup:manage_assigned

Layer	Technology	Purpose
Frontend	React (Next.js) + Tailwind CSS	Role-based dashboard, SSR for fast first paint
State	Redux Toolkit / Zustand + React Query	Global UI state + server-state caching
Backend	Node.js + Express.js	REST API, business logic
Database	MongoDB Atlas + Mongoose	Primary datastore, schema modeling
Cache	Redis	Rate limiting, refresh tokens, dashboard cache
Auth	JWT (access+refresh) + bcrypt	Stateless auth, RBAC claims
File storage	AWS S3 or Cloudinary	Brochures, creatives, documents

Deployment topology



Non-functional targets

Quality	Requirement
Security	bcrypt (cost≥10), short-lived JWT + rotating refresh, helmet, rate limiting
Tenant isolation	Server-side scopeToPartner() on every partner-data query
Scalability	Stateless API instances behind a load balancer; heavy exports run as background jobs
Reliability	MongoDB Atlas daily backups with point-in-time recovery; idempotent writes

PART B — LOW-LEVEL DESIGN (LLD)

Schema field-level detail, middleware/service design, API contracts, sequence flow — the "how"

LLD — Schema & Middleware Design

LLD-SCHEMA

Lead (Mongoose Schema)

partnerId: ObjectId (ref Partner, indexed) · courseId: ObjectId (ref Course) · assignedTo: ObjectId (ref User) · studentName, phone, whatsapp, email, city: String · source, priority, status: String (enum) · budget, leadScore: Number · followUpDate: Date · isDuplicate: Boolean · createdAt: Date

status enum: new | contacted | follow_up | demo | admitted | lost

pre-save hook: phone format validation · static duplicateCheck({partnerId, phone})

tenantScope.middleware.js

```
function scopeToPartner(req, res, next):
  if (req.user.role !== 'super_admin') req.query.partnerId = req.user.partnerId;
  next();
- injected on every route touching Lead, Admission, CommissionRecord, Notification
```

rbac.middleware.js

```
function requirePermission(permission):
  return (req, res, next) => {
    if (!req.user.role.permissions.includes(permission))
      return res.status(403).json({success:false, error:{code:'FORBIDDEN', ...}});
    next();
  }
```

CommissionService

+ calculateOnAdmission(admission): CommissionRecord — runs rule engine by course/partner category, writes earnedAmount + pendingAmount

+ requestPayout(commissionId, actor): CommissionRecord — payoutStatus → requested

+ approvePayout(commissionId, actor): CommissionRecord — Super Admin only; payoutStatus → approved → paid; writes AuditLog entry in the same operation

Indexing strategy (implementation detail)

```
db.leads.createIndex({ partnerId: 1, status: 1, followUpDate: 1 }) // dashboard pipeline view
db.leads.createIndex({ partnerId: 1, phone: 1 }) // duplicate detection
db.users.createIndex({ email: 1 }, { unique: true })
db.auditlogs.createIndex({ entityType: 1, timestamp: -1 })
```

POST /api/v1/leads

```
// Request body (Zod-validated)
{
  "studentName": "Aarav Sharma", "phone": "+91XXXXXXXXXX", "email": "aarav@example.com",
  "courseId": "665f...", "source": "website", "priority": "high", "budget": 45000
}

// Response 201
{ "success": true, "data": { "_id": "66a1...", "partnerId": "6512...", "status": "new",
  "leadScore": 62, "isDuplicate": false, "createdAt": "2026-08-28T10:12:00Z" } }

// Response 409 — duplicate detected
{ "success": false, "error": { "code": "DUPLICATE_LEAD",
  "message": "A lead with this phone number already exists for this partner" } }
```

PATCH /api/v1/commissions/{id}/payout

```
// Request — Super Admin only, requires permission "commission:approve_payout"
{ "action": "approve" }

// Response 200
{ "success": true, "data": { "_id": "...", "payoutStatus": "approved", "payoutDate": "2026-08-28T10:20:00Z" } }

// Response 403 — role lacks permission
{ "success": false, "error": { "code": "FORBIDDEN", "message": "role 'partner_admin' cannot approve payouts" } }
```

GET /api/v1/leads — list (paginated, tenant-scoped)

```
// Response 200
{ "success": true,
  "data": [ { "_id": "...", "studentName": "...", "status": "follow_up", ... } ],
  "meta": { "page": 1, "limit": 20, "total": 134 } }
// tenantScope.middleware.js has already injected partnerId into the Mongoose filter
// before the controller runs — Partner Admin/Team Member can never see another tenant's leads
```

- 1 Client → API — POST /api/v1/leads, header Authorization: Bearer <jwt>
- 2 auth.middleware.js — verifies JWT signature/expiry, attaches req.user = {id, role, partnerId, permissions}; 401 if invalid
- 3 rbac.middleware.js — requirePermission('lead:create') checks req.user.role.permissions; 403 if missing
- 4 tenantScope.middleware.js — injects req.body.partnerId = req.user.partnerId (skipped for Super Admin, who may specify any partnerId)
- 5 Zod validator — validates studentName, phone format, courseId shape; 400 on failure with field errors
- 6 LeadController.create() — calls LeadService.create(validatedBody)
- 7 LeadService — runs duplicateCheck({partnerId, phone}); if found, sets isDuplicate=true or returns 409 per business rule; computes leadScore
- 8 Mongoose — Lead.create(doc) — persisted to MongoDB Atlas
- 9 NotificationService — fires "lead assigned" notification to the owner (async, does not block the response)
- 10 Controller → Client — 201, { success: true, data: {...} }

Error → response-code mapping (this flow)

Condition	HTTP	error.code
Missing/expired JWT	401	UNAUTHENTICATED
Role lacks lead:create	403	FORBIDDEN
Invalid phone/email format	400	VALIDATION_ERROR
Duplicate {partnerId, phone}	409	DUPLICATE_LEAD

Two independent gates run before any business logic executes: **rbac.middleware.js** ("is this role allowed to create leads at all?") and **tenantScope.middleware.js** ("which tenant does this write belong to?"). This mirrors the SRS's core rule — a partner can access only its own tenant's data — enforced at the data-access layer, not the UI.